



PORTFOLIO

COMPUTACIONAL PHYSICS II

María Ignacia Wainer
CIENCIAS FÍSICAS
DEPARTAMENTO DE FÍSICA
Facultad de Ciencias Físicas y Matemáticas
UNIVERSIDAD DE CONCEPCIÓN

Agosto-Diciembre 2022
Concepción, Chile

Profesor Guía: Dr. Roberto Navarro Maldonado

Índice general

About this portfolio

This portfolio presents a collection of computational physics projects developed during the *Computational Physics II* course at the Department of Physics, Faculty of Physical and Mathematical Sciences, University of Concepción.

The work showcased in this portfolio focuses on the application of scientific computing techniques to the numerical solution of problems arising in physics and applied mathematics. Through the implementation of computational algorithms in Python, these projects explore fundamental topics such as numerical differentiation, numerical integration, root-finding methods, ordinary differential equations, interpolation, curve fitting, and stochastic simulations.

A central objective of this portfolio is to demonstrate the use of computational methods as practical tools for scientific investigation. The projects emphasize the development of programming skills, algorithmic thinking, numerical modeling, data analysis, and scientific visualization, all of which are essential components of modern computational research.

Throughout these activities, several widely used scientific computing tools were employed, including Python, NumPy, Pandas, Jupyter Notebook, Git, GitHub, and L^AT_EX. These technologies support reproducible research practices, version control, technical documentation, and the effective communication of scientific results.

The projects included in this portfolio reflect the application of numerical methods to a variety of physical and mathematical problems. In addition to strengthening programming proficiency, they provide practical experience in designing computational solutions, analyzing numerical accuracy, interpreting results, and communicating scientific findings through technical reports.

The skills developed through these projects form an important foundation for further work in computational physics, complex systems, data analysis, scientific modeling, and quantitative research.

Capítulo 1

Projects

1.1. Numerical Differentiation

In this activity, numerical methods for approximating derivatives were explored using Python. Several finite-difference schemes were studied, including forward, backward, and centered difference approximations. These methods provide practical tools for estimating derivatives when analytical expressions are unavailable or when working with discrete datasets.

This activity was completed individually during laboratory sessions.

1.1.1. Derivation of the Second-Order Finite Difference Approximation for the Second Derivative

The first part of the laboratory focused on deriving the second-order centered finite difference approximation for the second derivative,

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2). \quad (1.1)$$

To obtain this expression, the functions $f(x+h)$ and $f(x-h)$ were expanded using Taylor series around the point x , yielding

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{IV}(\xi^+) \quad (1.2)$$

$$f(x-h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 - \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{IV}(\xi^-). \quad (1.3)$$

Adding these two expressions eliminates the odd-order derivative terms, resulting in

$$f(x+h) + f(x-h) = 2f(x) + f''(x)h^2 + \frac{h^4}{4!} [f^{IV}(\xi^+) + f^{IV}(\xi^-)].$$

Solving for $f''(x)$ gives

$$f''(x) = \frac{f(x+h) + f(x-h) - 2f(x)}{h^2} - \frac{h^2}{4!} [f^{IV}(\xi^+) + f^{IV}(\xi^-)].$$

Since $x < \xi^+ < x+h$ and $x-h < \xi^- < x$, the second term represents the truncation error associated with the approximation. Defining

$$E(h) = -\frac{h^2}{4!} [f^{IV}(\xi^+) + f^{IV}(\xi^-)],$$

```

1  import numpy as np
2
3  def deriv2(f, x, h):
4      return (f(x + h) - 2 * f(x) + f(x - h)) / h**2
5
6  x = np.array([0.1, 1.0, np.pi/3])
7  h = np.geomspace(1, 1e-20, 100)
8  eps = 2**-52
9
10 # Evaluation for a chosen point (e.g., x = 1.0)
11 x_val = 1.0
12 num_res = deriv2(np.sqrt, x_val, h)
13 deriv2_an = -1 / (4 * x_val * np.sqrt(x_val))
14
15 error_num = np.abs(deriv2_an - num_res)
16 i = np.argmin(error_num)
17 optimal_h = h[i]

```

Código 1: Implementation of the numerical second derivative for $f(x) = \sqrt{x}$.

it follows that the error is proportional to h^2 , which leads to the well-known result

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2).$$

Therefore, the centered finite difference approximation provides a second-order accurate numerical estimate of the second derivative.

1.1.2. Implementation of the Numerical Second Derivative in Python

After deriving the finite-difference approximation, the next step was to implement Equation ?? in Python. A function, `deriv2(f, x, h)`, was created to compute the numerical approximation of the second derivative using the centered finite-difference scheme.

The implementation is shown in Code ??. The numerical approximation was evaluated for the function $f(x) = \sqrt{x}$ and compared against its analytical second derivative,

$$f''(x) = -\frac{1}{4x\sqrt{x}}.$$

The absolute error between the numerical and analytical solutions was then computed for different values of the step size h . To investigate the effect of discretization on numerical accuracy, a logarithmically spaced range of values for h was generated and evaluated.

A second script was used to visualize the numerical error as a function of the step size and to identify the value of h that minimizes the approximation error.

The results show that the numerical error depends strongly on the choice of the step size. For large values of h , truncation errors dominate the approximation, while for extremely small values of h , floating-point roundoff errors become significant. Consequently, an optimal value of h exists that minimizes the total numerical error.

Figure ?? illustrates the relationship between the approximation error and the step size. The minimum point of each curve corresponds to the value of h that provides the best balance between truncation and roundoff errors.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def deriv2(f, x, h):
5     return (f(x + h) - 2 * f(x) + f(x - h)) / h**2
6
7 def deriv2_an(x):
8     return -1 / (4 * x * np.sqrt(x))
9
10 x = np.array([0.1, 1, np.pi/3])
11 h = np.geomspace(0.1, 1e-20, 100)
12
13 for i in x:
14     error = np.abs(deriv2_an(i) - deriv2(np.sqrt, i, h))
15     plt.loglog(h, error, label=f'x = {i:.3f}', zorder=1)
16
17     j = np.argmin(error)
18     plt.scatter(h[j], error[j], marker="o", color="black", zorder=2)
19     print(f'Optimal h for x = {i:.3f}:', h[j])
20
21 plt.xlabel("Desplazamiento h")
22 plt.ylabel("Error")
23 plt.legend(loc='best', title='Valores fijos de x')
```

Código 2: Error analysis for the numerical approximation of the second derivative.

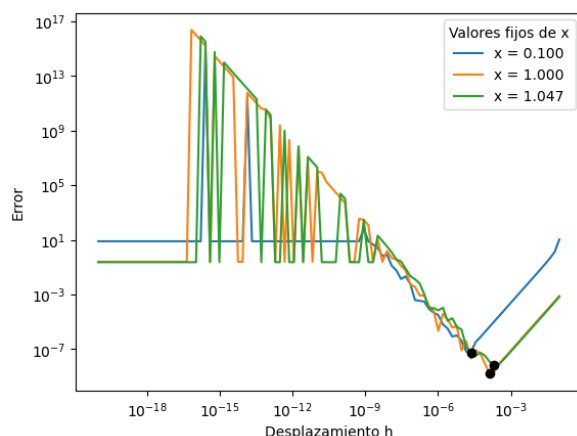


Figura 1.1: Relative error between the analytical and numerical second derivatives as a function of the step size h .

This exercise provided practical experience in implementing finite-difference methods and highlighted an important aspect of numerical analysis: increasing numerical precision is not simply a matter of choosing smaller step sizes. Instead, numerical accuracy depends on balancing competing sources of error, an idea that appears repeatedly in scientific computing and computational physics.

1.2. Git Version Control

This activity introduced the use of Git and GitHub as tools for version control, project management, and collaborative software development. The objective was to learn the fundamental workflows required to track changes, maintain project history, and manage source code repositories effectively.

This activity was completed individually during laboratory sessions.

1.2.1. Version Control and Repository Management with Git

The laboratory began with an introduction to GitHub and its role in modern software development and scientific computing. A personal repository was created to store and organize the projects developed throughout the course, providing a centralized platform for version control and documentation.

Next, several core Git commands were explored to interact with remote repositories. The command `git clone <repository_url>` was used to create a local copy of a remote repository. This workflow enables developers and researchers to work on projects locally while maintaining synchronization with a central repository.

The commands `git pull` and `git push` were then introduced. These commands allow users to retrieve updates from a remote repository and upload local changes, respectively.

A standard Git workflow was also practiced. First, files were staged using `git add`, which prepares modifications for inclusion in the next commit. Changes were then recorded using `git commit -m "message"`, where descriptive commit messages document the purpose of each modification. Finally, the updated project state was synchronized with the remote repository using `git push`.

Through this workflow, a complete version history of the project can be maintained, facilitating reproducibility, collaboration, and project organization.

1.2.2. Conclusion

This activity provided practical experience with Git and GitHub, two of the most widely used tools in software development, data science, and computational research.

Beyond their usefulness for managing coursework, version control systems play an essential role in professional and academic environments by enabling reproducible workflows, collaborative development, and systematic project documentation. The skills acquired in this laboratory establish a foundation for future work involving scientific programming, computational modeling, and collaborative research projects.

1.3. Numerical Integration.

This activity focused on the numerical evaluation of definite integrals using the Midpoint Rule. The objectives were to derive the method from Taylor series expansions, compare its numerical approximation with the analytical solution, and implement the algorithm in Python.

1.3.1. Derivation of the Midpoint Rule

The first part of the activity consisted of deriving the midpoint integration formula,

$$\int_a^b f(x) dx = hf\left(\frac{a+b}{2}\right) + O(h^3),$$

where $h = b - a$ and the function is evaluated at the midpoint of the integration interval.

To derive this expression, the function was expanded around the midpoint

$$M = \frac{a+b}{2}$$

using a Taylor series. Substituting the expansion into the integral and integrating term by term yields an approximation in which the leading contribution is proportional to the function evaluated at the midpoint.

The remaining higher-order terms constitute the truncation error of the method, which scales as $O(h^3)$. Therefore, the midpoint rule provides a second-order accurate approximation of the definite integral.

1.3.2. Analytical and Numerical Evaluation

The midpoint rule was applied to the integral

$$I = \int_0^3 (x-1)e^x dx, \tag{1.4}$$

resulting in the approximation

$$I \approx \frac{3}{2}e^{3/2}. \tag{1.5}$$

To assess the accuracy of the numerical method, the exact analytical solution was also computed,

$$I = e^3 + 2. \tag{1.6}$$

Comparing Equations ?? and ?? reveals the approximation error associated with evaluating the integral using a single midpoint. This difference illustrates the limitations of low-order numerical quadrature when applied over large intervals.


```

1 import numpy as np
2
3 def midpoint_simple(f, a, b):
4     return (b - a) * f((a + b) / 2)
5
6 def g(x):
7     return (x - 1) * np.exp(x)
8
9 # Numerical evaluation
10 result = midpoint_simple(g, 0, 3)

```

Código 3: Implementation of the Midpoint Rule.

```

1 lim = np.linspace(0, 3, 100)
2 int_0 = 0.0
3
4 for i in range(len(lim) - 1):
5     int_0 += midpoint_simple(g, lim[i], lim[i+1])

```

Código 4: Composite Midpoint Rule implementation.

1.3.3. Python Implementation

The midpoint rule was implemented in Python through the function `midpoint_simple(f,a,b)`, shown in Code ??.

The numerical result obtained from the implementation matched the value predicted by Equation ??, confirming the correctness of the algorithm.

1.3.4. Composite Midpoint Rule

To improve the approximation, the integration interval was subdivided into multiple smaller intervals,

$$\int_a^b f(x) dx = \int_a^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \cdots + \int_{x_N}^b f(x) dx.$$

The midpoint rule was then applied independently to each subinterval. This procedure is known as the *Composite Midpoint Rule*. The implementation is shown in Code ??.

Using multiple subintervals produced a more accurate numerical approximation while maintaining the simplicity of the original method. This demonstrates a common strategy in numerical analysis: improving accuracy by refining the discretization of the problem.

1.3.5. Discussion

This activity provided practical experience with numerical integration and highlighted the relationship between analytical and computational approaches to solving mathematical problems.

The implementation of both the standard and composite midpoint rules demonstrated how numerical quadrature methods can approximate definite integrals efficiently while offering controllable

accuracy through interval subdivision. These concepts form the foundation of more advanced numerical integration techniques used in scientific computing, computational physics, and data analysis.

1.4. Root Finding Methods

This activity focused on the numerical determination of equilibrium points through root-finding techniques. The main objectives were to compute numerical derivatives, analyze the behavior of a parameter-dependent function, and locate its critical points using the bisection method.

These techniques are fundamental in computational physics and scientific computing, where analytical solutions are often unavailable and numerical methods must be used to identify equilibria and characterize system behavior.

1.4.1. Numerical Differentiation and Root Finding

The first task consisted of analyzing the function

$$u(\theta) = \cos(\theta) + \beta \sin^2(\theta), \quad (1.7)$$

for values of θ in the interval

$$0 \leq \theta \leq \frac{\pi}{2},$$

and for ten different values of the parameter β between 0 and 1.

The derivative of the function was approximated numerically using a centered finite-difference scheme,

$$u'(\theta) \approx \frac{u(\theta + h) - u(\theta - h)}{2h},$$

which provides second-order accuracy with respect to the step size h .

After computing the derivative, the bisection method was used to determine the values of θ satisfying

$$u'(\theta) = 0.$$

These solutions correspond to critical points of the function and may represent equilibrium configurations of the underlying physical system. The implementation is shown in Code ??.

1.4.2. Results

Figure ?? presents the numerical derivative for different values of β in the interval $[0, 1]$.

The zeros of the derivative correspond to locations where the slope changes sign and therefore indicate potential equilibrium points. As the parameter β varies, the number and position of these critical points change, revealing a dependence of the system's behavior on the control parameter.

Figure ?? focuses on the regime $\beta > 1/2$, where additional equilibrium solutions emerge.

The corresponding equilibrium angles obtained through the bisection algorithm are shown in Figure ??.

1.4.3. Discussion

This activity demonstrated how numerical differentiation and root-finding algorithms can be combined to identify equilibrium states in parameter-dependent systems.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, np.pi/2, 100)
5
6 for b in np.linspace(0, 1, 10):
7     def u(theta):
8         return np.cos(theta) + b * (np.sin(theta))**2
9
10    def du(theta, h=1e-3):
11        return (u(theta + h) - u(theta - h)) / (2 * h)
12
13    plt.plot(x, du(x))
14
15    # Root-finding using the bisection method for the last system state
16    m = 0.1
17    n = 1.25
18
19    for i in range(10):
20        c = 0.5 * (m + n)
21        condicion = du(m) * du(c)
22
23        if condicion <= 0:
24            n = c
25        else:
26            m = c
27
28    plt.plot(c, du(c), 'o')
```

Código 5: Numerical differentiation and root finding using the bisection method.

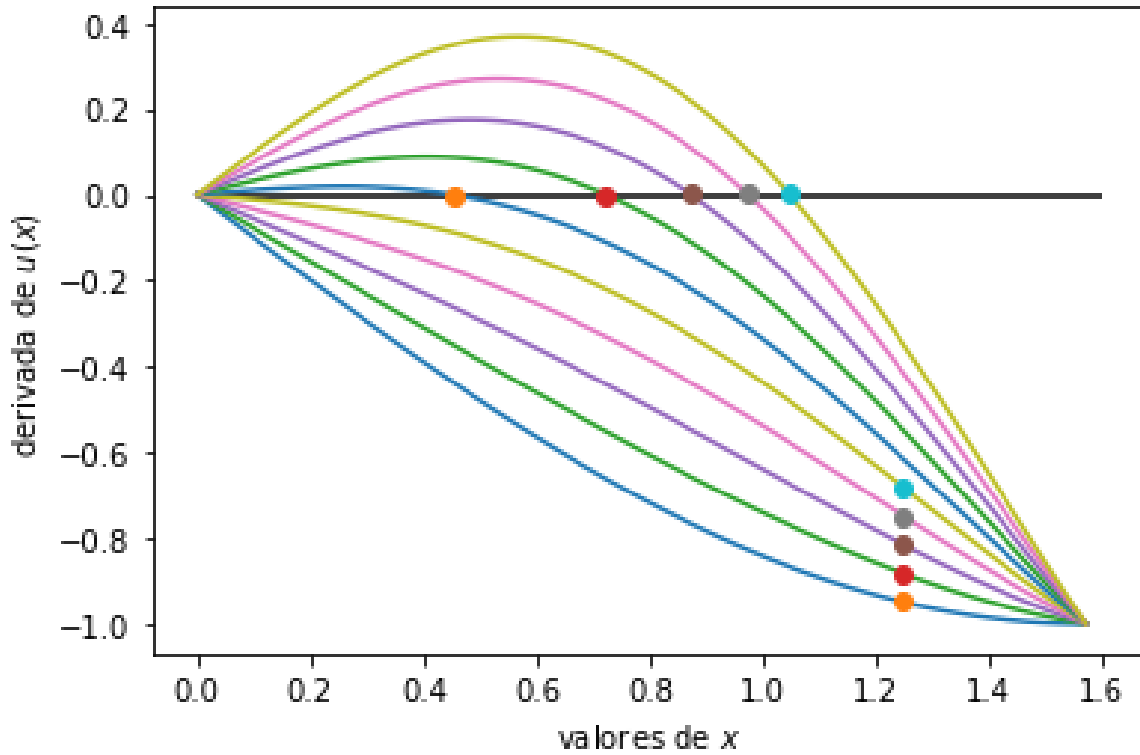


Figura 1.2: Numerical derivative of $u(\theta)$ for multiple values of β .

The bisection method proved to be a reliable technique for locating the roots of the derivative, while the graphical analysis provided insight into how the equilibrium configurations evolve as the parameter β changes.

Beyond this specific example, root-finding methods play a central role in scientific computing and are widely used in optimization, nonlinear dynamics, engineering simulations, and computational physics. The techniques implemented here form part of the core numerical toolkit required for the analysis of mathematical models and dynamical systems.

1.5. Ordinary Differential Equations

This activity focused on the numerical study of the Lotka–Volterra predator–prey model, a classical system of nonlinear differential equations widely used in population dynamics and complex systems research.

The objectives were to derive a dimensionless form of the equations, implement a numerical solver based on the Euler–Cromer method, investigate the influence of different initial conditions on the system dynamics, and verify the conservation of an invariant quantity associated with the model.

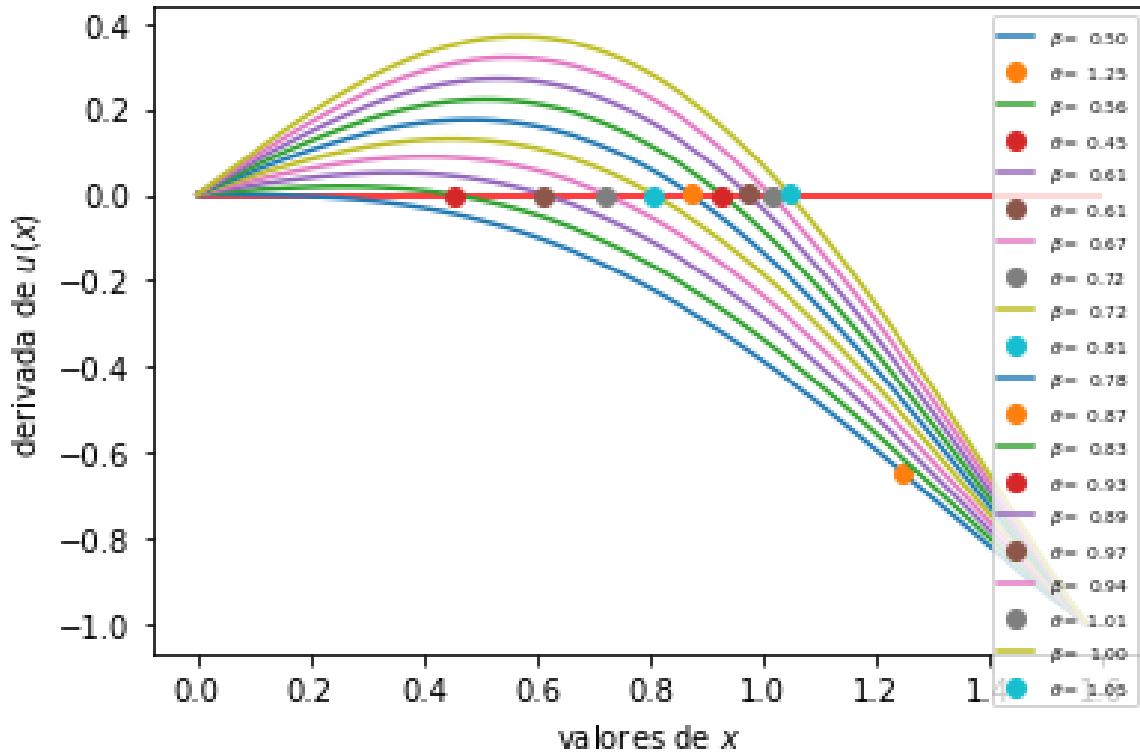


Figura 1.3: Numerical derivative for values of β greater than $1/2$.

1.5.1. Dimensionless Form of the Lotka–Volterra Equations

The Lotka–Volterra predator–prey model is given by

$$\begin{aligned}\frac{dP}{dt} &= \alpha P - \beta PD, \\ \frac{dD}{dt} &= -\gamma D + \delta PD,\end{aligned}\tag{1.8}$$

where P and D represent the prey and predator populations, respectively.

The first part of the activity consisted of showing that the system can be rewritten in a dimensionless form through an appropriate change of variables. Introducing scaled variables Π , Δ , and the dimensionless time τ , the equations become

$$\frac{d\Pi}{d\tau} = \Pi(1 - \Delta),\tag{1.9}$$

and

$$\frac{d\Delta}{d\tau} = \mu\Delta(\Pi - 1),\tag{1.10}$$

where μ is a dimensionless parameter that characterizes the relative growth and decay rates of the interacting populations.

This reformulation simplifies the mathematical structure of the model while preserving its dynamical behavior, making it more convenient for numerical analysis.

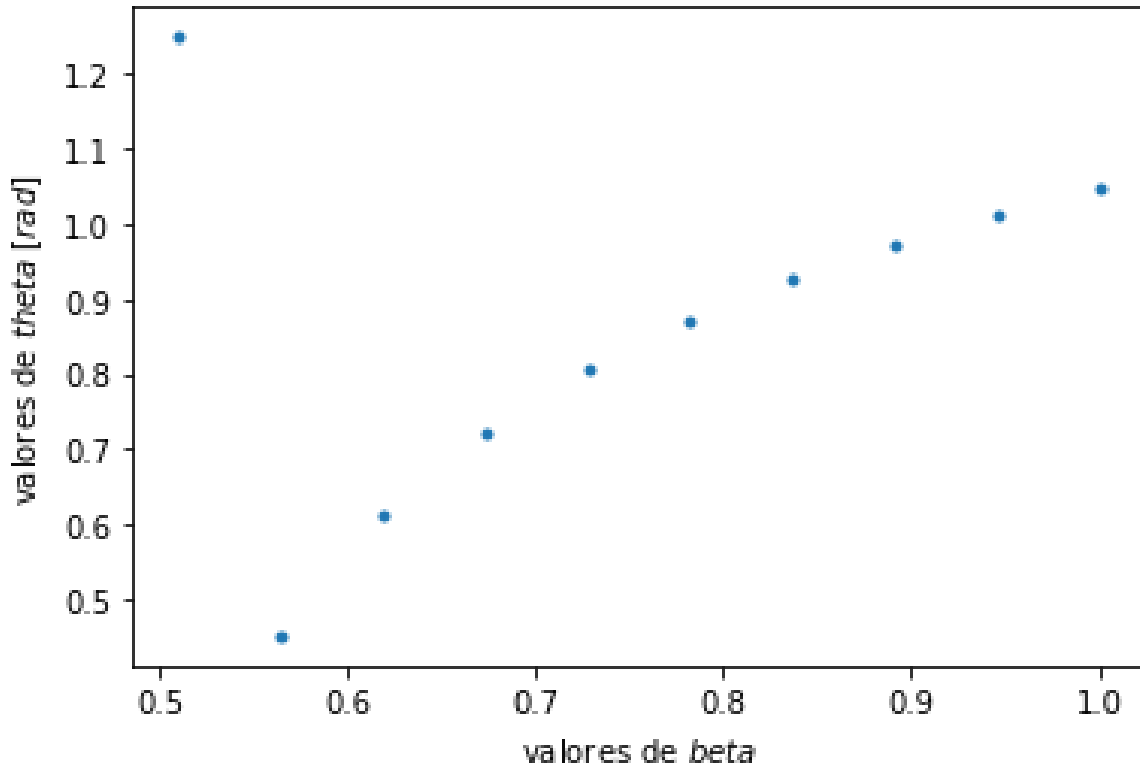


Figura 1.4: Equilibrium angle θ as a function of the parameter β .

1.5.2. Numerical Solution Using the Euler–Cromer Method

The dimensionless system was solved numerically using the Euler–Cromer method. Time was discretized with a constant step size, and the population variables were updated iteratively according to the governing equations. The implementation is shown in Code ??.

Figure ?? shows the resulting population dynamics. The oscillatory behavior is characteristic of predator–prey systems: an increase in the prey population is followed by delayed predator growth, which subsequently reduces the prey population. As predator numbers decrease, prey populations recover, generating a repeating cycle.

1.5.3. Phase-Space Analysis

The numerical experiment was repeated for multiple initial values of $\Delta(0)$ while keeping the remaining parameters fixed. The corresponding implementation is shown in Code ??.

The resulting trajectories are displayed in Figure ??.

The phase-space trajectories form closed orbits around the equilibrium point, indicating periodic behavior and demonstrating the stability properties of the system. This representation provides a useful visualization of how predator and prey populations evolve relative to one another.

```
1 import numpy as np
2
3 t = np.arange(0, 40, 0.1)
4 h = 0.1
5
6 pi = np.zeros(len(t))
7 delta = np.zeros(len(t))
8
9 mu = 1
10 pi[0] = 0.1
11 delta[0] = 5.0
12
13 for n in range(len(t) - 1):
14     pi[n+1] = pi[n] + h * pi[n] * (1 - delta[n])
15     delta[n+1] = delta[n] + h * mu * delta[n] * (pi[n+1] - 1)
```

Código 6: Euler–Cromer solution of the Lotka–Volterra system.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 mu = 1
5 t = np.arange(0, 100, 0.1)
6 h = 0.05
7 pi = np.zeros(len(t))
8 delta = np.zeros(len(t))
9
10 for Delta0 in np.arange(1.1, 10, 1):
11     pi[0] = 1.0
12     delta[0] = Delta0
13     for n in range(len(t) - 1):
14         pi[n+1] = pi[n] + h * pi[n] * (1 - delta[n])
15         delta[n+1] = delta[n] + h * mu * delta[n] * (pi[n+1] - 1)
16     plt.plot(delta, pi)
```

Código 7: Phase-space trajectories for different initial conditions.

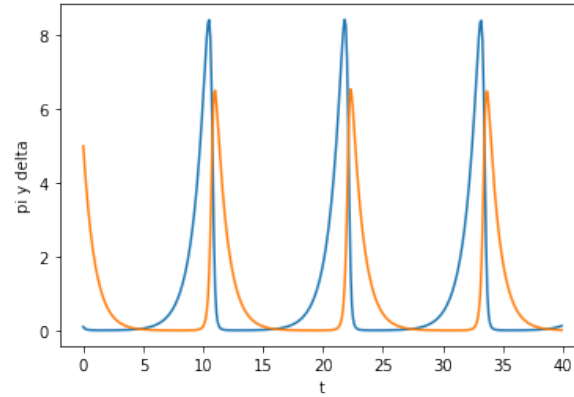


Figura 1.5: Time evolution of predator and prey populations.

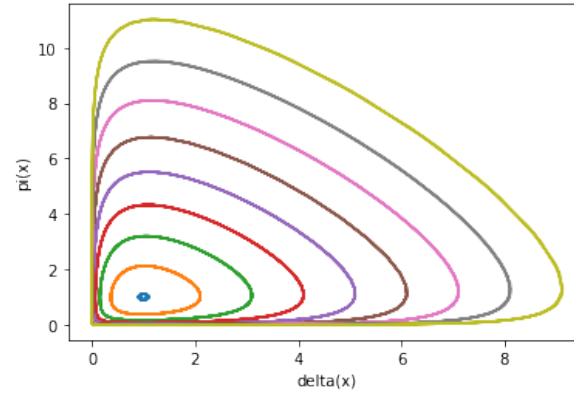


Figura 1.6: Phase-space representation of the Lotka–Volterra dynamics.

1.5.4. Verification of the Conserved Quantity

One of the most important properties of the Lotka–Volterra model is the existence of an invariant quantity,

$$C = \ln \Delta - \Delta + \mu(\ln \Pi - \Pi), \quad (1.11)$$

which remains constant along exact trajectories of the system.

To verify the accuracy of the numerical solution, the quantity C was evaluated throughout the simulation using the implementation shown in Code ??.

The resulting graph is presented in Figure ??.

The near-constant behavior of C throughout the simulation indicates that the Euler–Cromer method reproduces the dynamics of the system with good numerical accuracy. Significant deviations from constancy would suggest the accumulation of numerical errors and a loss of fidelity in the computed trajectories.

1.5.5. Discussion

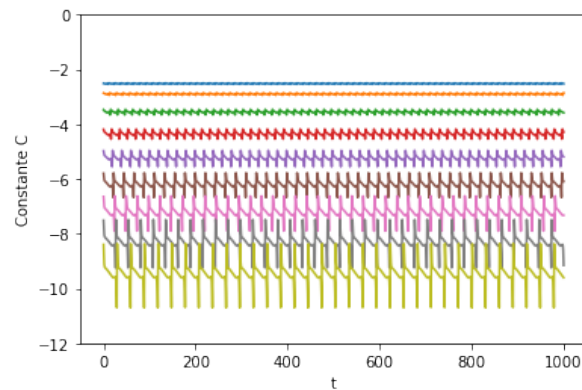
This activity provided practical experience in the numerical solution of nonlinear dynamical systems and demonstrated several important concepts in computational physics, including dimensionless


```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 mu = 1
5 t = np.arange(0, 1000, 0.1)
6 h = 0.05
7 pi = np.zeros(len(t))
8 delta = np.zeros(len(t))
9
10 for Delta0 in np.arange(1.1, 10, 1):
11     pi[0] = 0.3
12     delta[0] = Delta0
13     for n in range(len(t) - 1):
14         pi[n+1] = pi[n] + h * pi[n] * (1 - delta[n])
15         delta[n+1] = delta[n] + h * mu * delta[n] * (pi[n+1] - 1)
16
17     C = np.log(delta) - delta + mu * (np.log(pi) - pi)
18     plt.plot(t, C)

```

Código 8: Evaluation of the Lotka–Volterra invariant.

Figura 1.7: Numerical evaluation of the conserved quantity C .

analysis, numerical integration of differential equations, phase-space visualization, and conservation laws.

The Lotka–Volterra model serves as a classical example of how relatively simple nonlinear equations can generate rich dynamical behavior. The numerical experiments performed here illustrate the usefulness of computational methods for studying systems that are difficult to analyze analytically and provide a foundation for more advanced work in complex systems, mathematical modeling, and population dynamics.

```

1 import numpy as np
2
3 def f(u, t, o=10, p=28, b=8/3):
4     x, y, z = u
5     return np.array([o * (y - x), p * x - y - x * z, x * y - b * z])
6
7 tmax = 30.0
8 h = 0.01
9 t = np.arange(0, tmax, h)
10
11 u = np.empty([t.size, 3])
12 u[0] = [0.0, 1.0, 1.05] # Initial conditions
13
14 for n in range(t.size - 1):
15     K1 = f(u[n], t[n])
16     K2 = f(u[n] + 0.5 * h * K1, t[n] + 0.5 * h)
17     K3 = f(u[n] + 0.5 * h * K2, t[n] + 0.5 * h)
18     K4 = f(u[n] + h * K3, t[n] + h)
19     u[n+1] = u[n] + (h / 6) * (K1 + 2 * K2 + 2 * K3 + K4)

```

Código 9: Fourth-order Runge–Kutta solution of the Lorenz system.

1.6. Runge Kutta Methods

This activity explored the numerical solution of a nonlinear dynamical system using the fourth-order Runge–Kutta (RK4) method. The motivation was inspired by the famous question often associated with chaos theory:

“Could the flap of a butterfly’s wings in Brazil set off a tornado in Texas?”

This idea, known as the *Butterfly Effect*, reflects the sensitivity of certain nonlinear systems to small changes in their initial conditions.

The objective of this activity was to solve the Lorenz system numerically, investigate the influence of different initial conditions on the resulting trajectories, and visualize the system’s chaotic behavior in both two- and three-dimensional representations.

1.6.1. Numerical Solution of the Lorenz System

The Lorenz equations are given by

$$\begin{aligned}
 \dot{x} &= \sigma(y - x), \\
 \dot{y} &= \rho x - y - xz, \\
 \dot{z} &= xy - \beta z,
 \end{aligned} \tag{1.12}$$

where σ , ρ , and β are model parameters.

To solve this system numerically, a fourth-order Runge–Kutta integrator was implemented. The RK4 method improves accuracy by evaluating the derivatives at four intermediate points during each integration step. These evaluations are represented by the quantities K_1 , K_2 , K_3 , and K_4 . The implementation is shown in Code ??.

The simulation was performed over a finite time interval using a constant time step, allowing the trajectory of the system to be reconstructed numerically.

1.6.2. Analysis of Sensitivity to Initial Conditions

The first simulation used the initial conditions provided in the laboratory instructions. The resulting trajectory is shown in Figure ??.

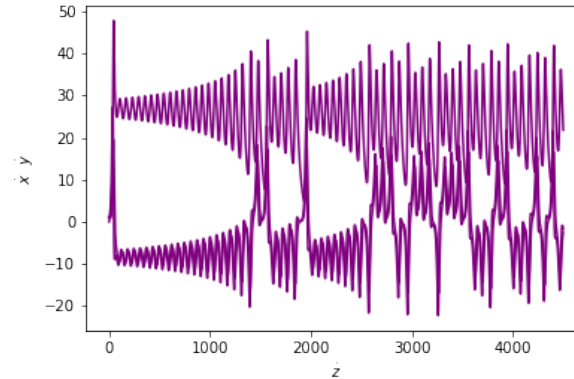


Figura 1.8: Lorenz system trajectory obtained using the original initial conditions.

A second simulation was then performed using slightly modified initial conditions,

$$x(0) = 1,0, \quad y(0) = 1,5, \quad z(0) = 0,05.$$

The resulting trajectory is shown in Figure ??.

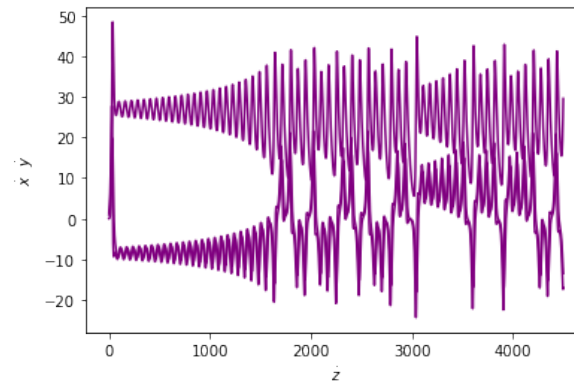


Figura 1.9: Lorenz system trajectory obtained using perturbed initial conditions.

Although the changes in the initial conditions were relatively small, the long-term behavior of the trajectories became significantly different. This sensitivity to initial conditions is one of the defining characteristics of chaotic systems and illustrates why long-term prediction becomes difficult in many nonlinear dynamical models.

1.6.3. Three-Dimensional Visualization

To better visualize the geometry of the system, the numerical solution was also represented in three-dimensional phase space. The resulting trajectory is shown in Figure ??.

The characteristic butterfly-shaped structure emerges naturally from the simulation and corresponds to the well-known Lorenz attractor. Despite the deterministic nature of the governing equations, nearby trajectories diverge rapidly, producing complex and apparently unpredictable behavior.

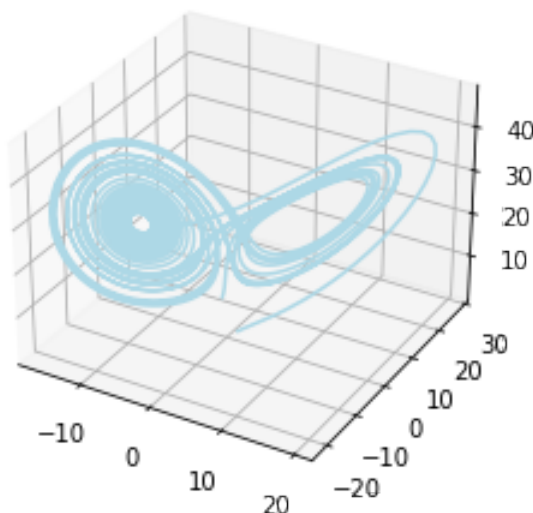


Figura 1.10: Three-dimensional phase-space trajectory of the Lorenz system.

1.6.4. Discussion

This activity provided practical experience with the numerical integration of nonlinear differential equations and introduced several key concepts in dynamical systems theory and chaos.

The fourth-order Runge–Kutta method proved to be an effective tool for accurately solving the Lorenz equations, while the comparison of nearby trajectories demonstrated the phenomenon of sensitive dependence on initial conditions. The resulting Lorenz attractor illustrates how deterministic systems can generate highly complex behavior without the presence of randomness.

These ideas form the foundation of modern studies in nonlinear dynamics, complex systems, climate modeling, fluid dynamics, and many other areas of computational science.

1.7. Curve Fitting

1.7.1. Analysis of Voyager Mission Magnetic Field Data

In this activity, magnetic field measurements from NASA’s Voyager mission were analyzed using numerical fitting and interpolation techniques. The objective was to explore different approaches for modeling and reconstructing experimental data while comparing their respective characteristics and applications.

A linear model of the form

$$p(x) = a_0 + a_1x,$$

where $x = t - t_0$, was fitted to the dataset in order to identify the overall trend of the measurements. To perform this task, a custom polynomial fitting routine was implemented, as shown in Code ???. The fitting coefficients were obtained through a least-squares approach based on the normal equations.

The linear fit was then applied to the Voyager dataset using the code shown in Code ??.

In addition to the linear regression, a cubic spline interpolation was performed. Unlike regression models, spline interpolation constructs piecewise cubic polynomials that pass through every data point while maintaining smoothness between intervals. The interpolation was implemented using the `cspline` function shown in Code ??.

```

1 class polyfit(PolyClass.Poly):
2     def __init__(self, x, y, grado=1):
3         G = np.vander(x, grado + 1, increasing=True)
4
5         M = G.T @ G
6         z = G.T @ y
7
8         self.coef = Resolver(M, z)
9         self.chi2 = np.sum((G @ self.coef - y)**2)

```

Código 10: Implementation of a polynomial least-squares fitting routine.

```

1 x = xi - xi[0]
2 f = polyfit(x, yi)

```

Código 11: Linear fit applied to the Voyager magnetic field data.

The results of both approaches are presented in Figures ?? and ?. Figure ?? shows the linear fit superimposed on the original measurements, while Figure ?? displays the smooth curve generated by the cubic spline interpolation.

1.7.2. Discussion

This activity highlights the fundamental difference between regression and interpolation techniques. The linear fit captures the overall trend of the data by minimizing the fitting error, making it useful for identifying large-scale behavior and obtaining simplified models.

In contrast, cubic spline interpolation aims to reproduce the dataset exactly by constructing a smooth curve that passes through every observation. As a result, interpolation provides a more detailed representation of the measured values, whereas regression focuses on extracting underlying trends.

Through this exercise, practical experience was gained in scientific data analysis, least-squares optimization, interpolation methods, and the visualization of experimental datasets. These techniques are widely used in computational physics, data science, and quantitative research for the analysis and interpretation of observational data.

1.8. Interpolation Methods

This activity focused on the interpolation of historical sunspot data provided by the instructor. The objective was to reconstruct the behavior of solar activity over time and compare different interpolation techniques commonly used in scientific computing.

```

1 g = cspline(xi, yi)

```

Código 12: Cubic spline interpolation of the Voyager dataset.

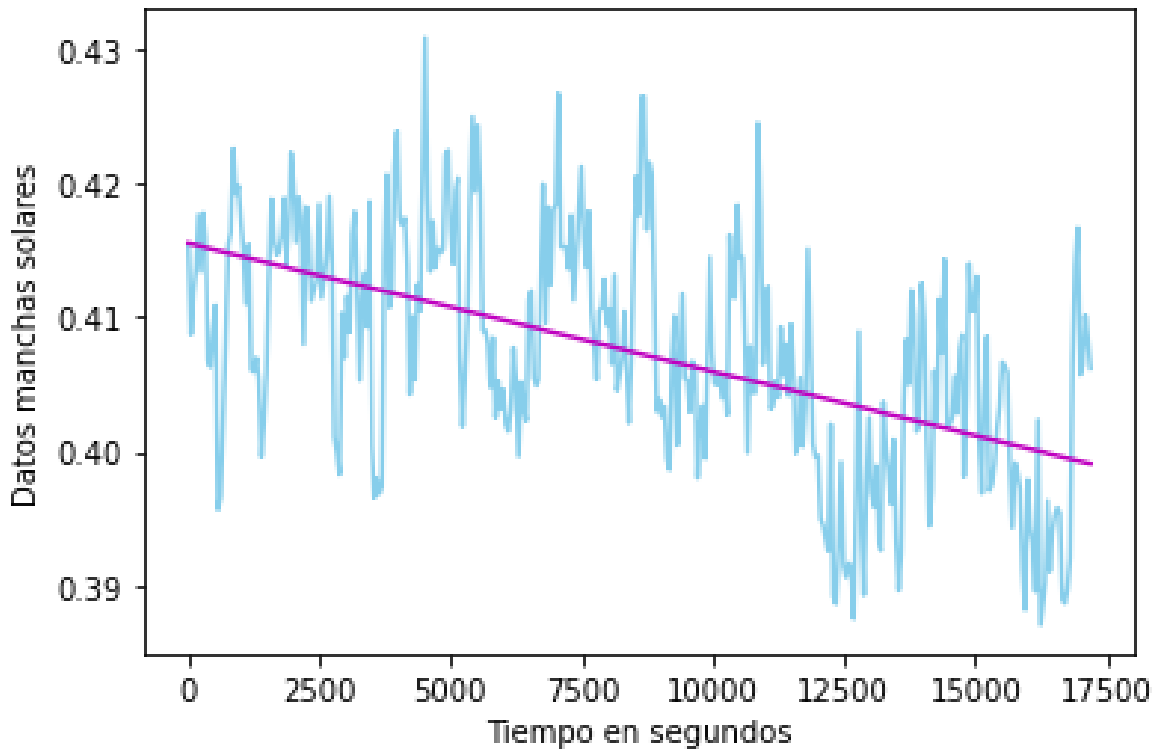


Figura 1.11: Voyager magnetic field data with a linear least-squares fit.

Three interpolation methods were investigated:

- Lagrange polynomial interpolation.
- Cubic spline interpolation.
- Polynomial interpolation using Vandermonde matrices.

The resulting interpolations were then compared in terms of accuracy, smoothness, and implementation complexity.

1.8.1. Interpolation Methods

The first approach employed Lagrange polynomial interpolation. A custom implementation of the Lagrange basis polynomials was developed, as shown in Code ???. To improve numerical stability and avoid oscillatory behavior associated with high-degree polynomials, the dataset was divided into smaller intervals containing six data points each. The interpolation was then performed independently within each interval.

The second method utilized cubic spline interpolation. In this case, the `cspline` function from the `SplineInterpolation` library was applied directly to the dataset. Cubic splines construct piecewise cubic polynomials while maintaining continuity of the function and its derivatives across interval boundaries.

Finally, polynomial interpolation based on Vandermonde matrices was implemented. Similar to the Lagrange approach, the dataset was divided into smaller subsets to reduce numerical instabilities

```
1 def f(x, xi, yi):
2     pol = 0.0
3     N = len(xi)
4     for i in range(N):
5         Li = yi[i]
6         for j in range(N):
7             if j != i:
8                 Li = Li * (x - xi[j]) / (xi[i] - xi[j])
9         pol = pol + Li
10    return pol
11
12 for i in range(0, len(xi), 5):
13     xi2 = xi[i:i+6]
14     yi2 = yi[i:i+6]
15
16     x = np.linspace(xi2.min(), xi2.max(), 50)
17     y = f(x, xi2, yi2)
18
19     plt.scatter(xi2, yi2)
20     plt.plot(x, y)
```

Código 13: Lagrange polynomial interpolation.

```
1 g = cspline(xi, yi)
```

Código 14: Cubic spline interpolation.

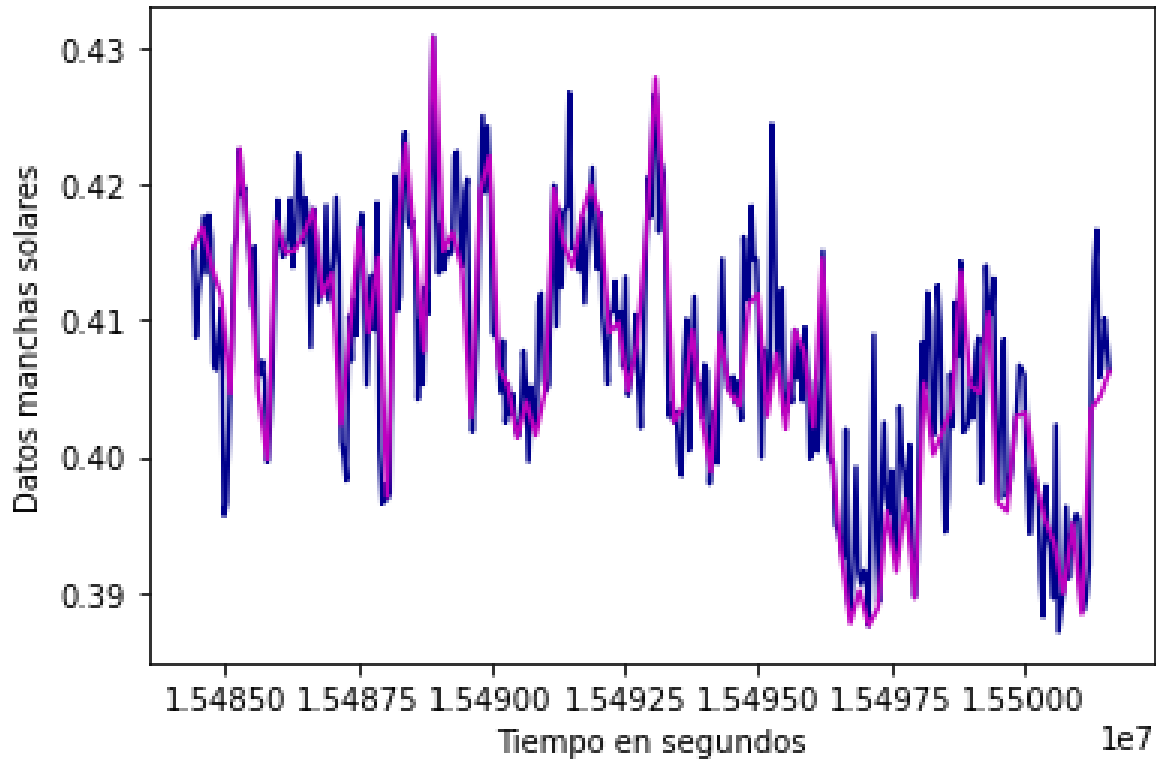


Figura 1.12: Voyager magnetic field data reconstructed using cubic spline interpolation.

associated with high-degree polynomial interpolation. For each interval, the interpolation coefficients were obtained by solving the corresponding Vandermonde system.

1.8.2. Results and Visualization

Figures ??, ??, and ?? show the interpolated sunspot data obtained using Lagrange polynomials, cubic splines, and Vandermonde matrices, respectively.

Visual inspection of the results indicates that all three methods successfully reproduce the observed behavior of the dataset. Although minor differences appear between the approaches, the overall trends remain consistent across the interpolated curves.

1.8.3. Discussion

This activity provided an opportunity to compare multiple interpolation techniques applied to the same scientific dataset. While all three methods produced similar results, important differences emerged in terms of implementation and computational efficiency.

The cubic spline approach proved particularly convenient because it could be applied directly to the complete dataset without the need to partition the data into smaller intervals. In contrast, both the Lagrange and Vandermonde methods required segmentation of the dataset to maintain numerical stability and avoid artifacts associated with high-degree polynomial interpolation.

Beyond the specific application to sunspot observations, this exercise illustrates how interpolation techniques can be used to reconstruct continuous behavior from discrete measurements. Such methods are widely employed in computational physics, data analysis, signal processing, and scientific modeling.


```

1 for i in range(0, len(xi), 3):
2     xi2 = xi[i:i+4]
3     yi2 = yi[i:i+4]
4
5     x = np.linspace(xi2.min(), xi2.max(), 50)
6     f2 = Vandermonde(xi2, yi2)
7     y = f2(x)
8
9     plt.scatter(xi2, yi2)
10    plt.plot(x, y)

```

Código 15: Interpolation using Vandermonde matrices.

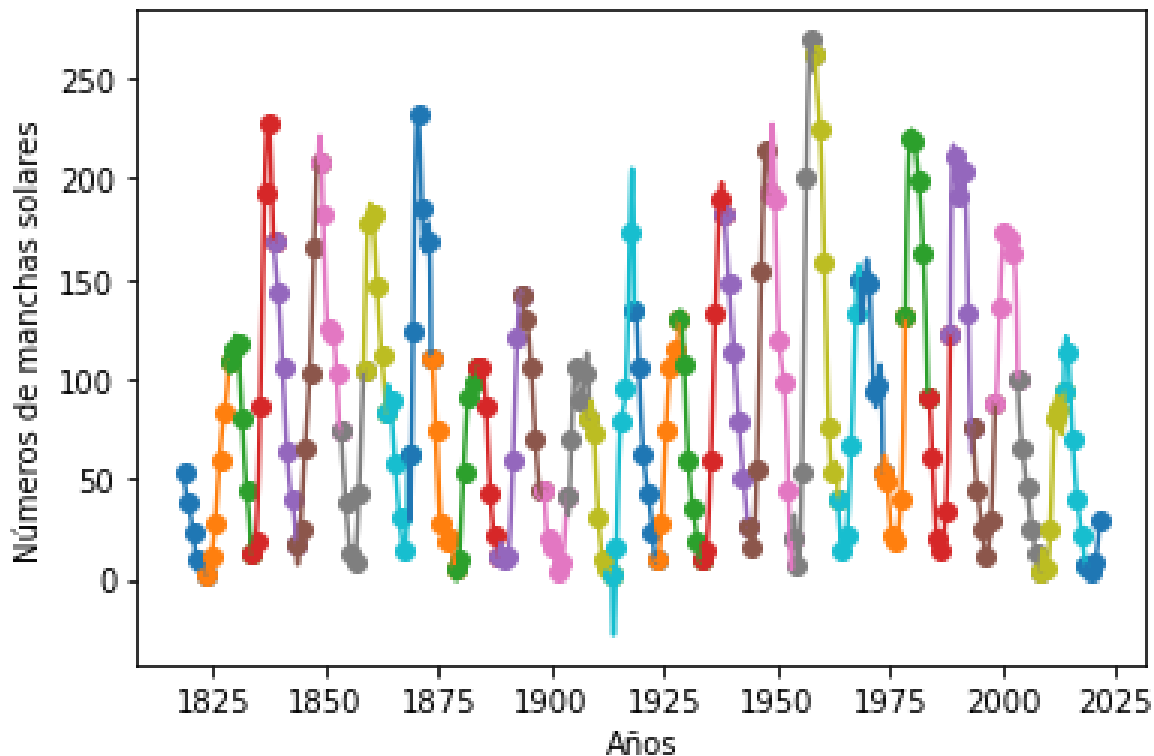


Figura 1.13: Interpolation of sunspot data using Lagrange polynomials.

1.9. Stochastic Simulations.

This activity explored the numerical estimation of the mathematical constant π using a Monte Carlo simulation. The method is based on a geometric relationship between the area of a circle and the area of the square in which it is inscribed.

The objectives were to derive the analytical relationship used to estimate π , implement a random sampling algorithm in Python, and investigate how probabilistic methods can be used to approximate

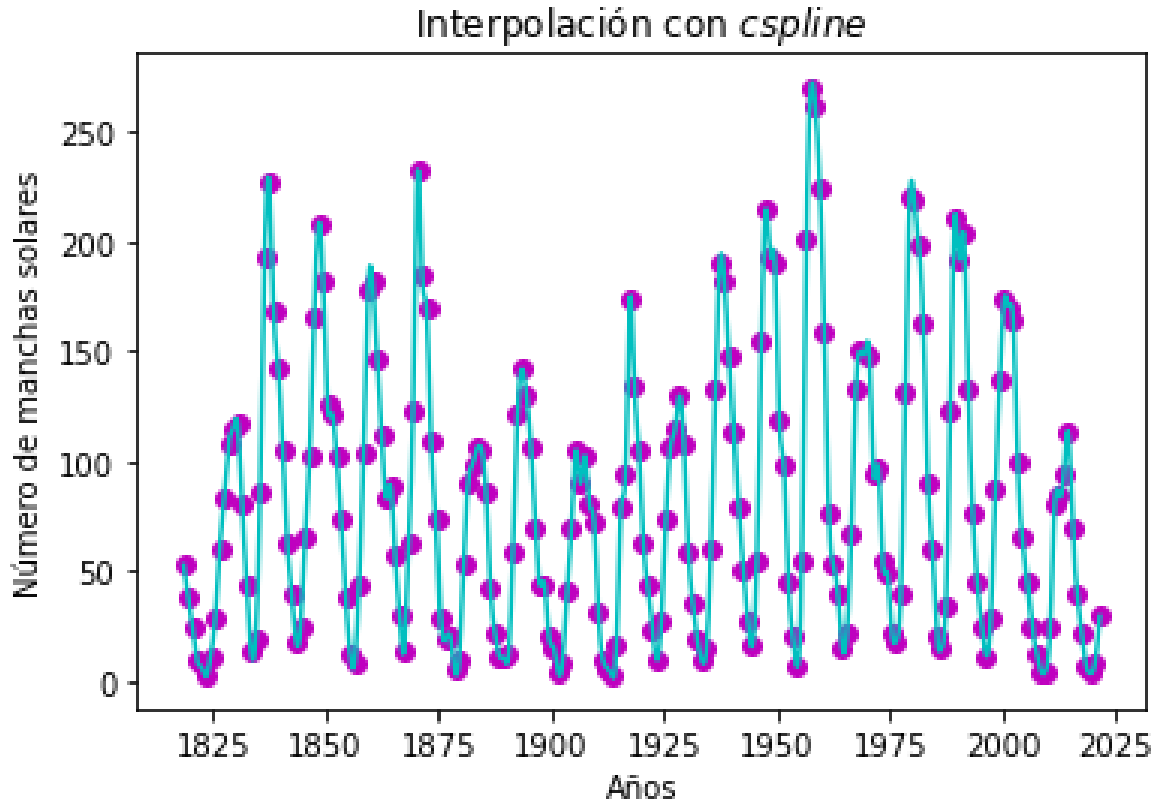


Figura 1.14: Interpolation of sunspot data using cubic splines.

mathematical quantities.

1.9.1. Monte Carlo Estimation of π

Consider a circle of radius $r = \frac{1}{2}$ inscribed in a square of side length 1. The areas of the two geometric figures are

$$A_{\text{circle}} = \pi r^2 = \frac{\pi}{4},$$

and

$$A_{\text{square}} = 1.$$

Therefore,

$$\pi = 4 \frac{A_{\text{circle}}}{A_{\text{square}}}.$$

This relationship forms the basis of the Monte Carlo approach. If points are distributed uniformly at random within the square, the fraction of points that fall inside the circle approximates the ratio between the two areas.

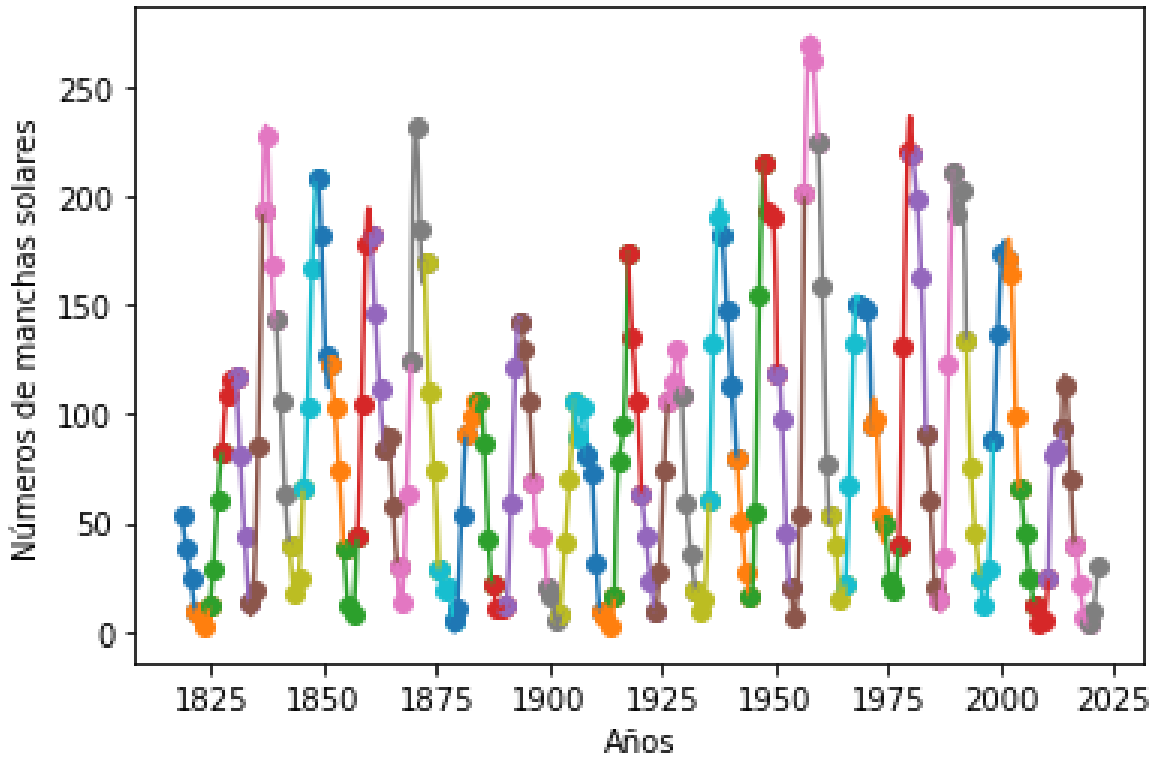


Figura 1.15: Interpolation of sunspot data using Vandermonde matrices.

1.9.2. Implementation

A function was first created to generate random points inside the square. Each point was then classified according to whether it lay inside or outside the inscribed circle. A point belongs to the circle if

$$x^2 + y^2 < r^2.$$

The Monte Carlo estimator for π is then given by

$$\pi \approx 4 \frac{N_{\text{inside}}}{N_{\text{total}}},$$

where N_{inside} denotes the number of points inside the circle and N_{total} is the total number of generated points. The complete implementation is shown in Code ??.

The simulation was performed using 100,000 randomly generated points. In addition to estimating π , the algorithm stored the coordinates of points inside and outside the circle to visualize the sampling process.

1.9.3. Results

Figure ?? shows the distribution of sampled points within the square. Points falling inside the circle are displayed separately from those outside the circle, providing a visual confirmation of the classification procedure.

```
1 import numpy as np
2
3 N_total = 100000
4
5 # Generating uniform random coordinates within the square bounds [-0.5, 0.5]
6 x = np.random.uniform(-0.5, 0.5, N_total)
7 y = np.random.uniform(-0.5, 0.5, N_total)
8
9 # Distance condition to check if points fall inside the circle (r = 0.5)
10 inside_circle = (x**2 + y**2) < 0.25
11
12 N_inside = np.sum(inside_circle)
13
14 # Monte Carlo Pi approximation
15 pi_estimate = 4 * N_inside / N_total
16 print(f"Estimated Pi value: {pi_estimate}")
```

Código 16: Monte Carlo estimation of π .

As the number of sampled points increases, the estimated value converges toward the true value of π . This behavior is a direct consequence of the Law of Large Numbers, which guarantees that the empirical area ratio approaches the theoretical ratio as the sample size grows.

1.9.4. Discussion

This activity provided an introduction to Monte Carlo methods, a class of computational techniques that use randomness to solve mathematical and physical problems.

Although estimating π is a simple example, the same principles are widely applied in computational physics, quantitative finance, statistical inference, machine learning, uncertainty quantification, and large-scale simulations where deterministic solutions may be impractical or unavailable.

The exercise demonstrates how random sampling can be used to approximate geometric quantities and highlights the power of probabilistic algorithms in scientific computing.

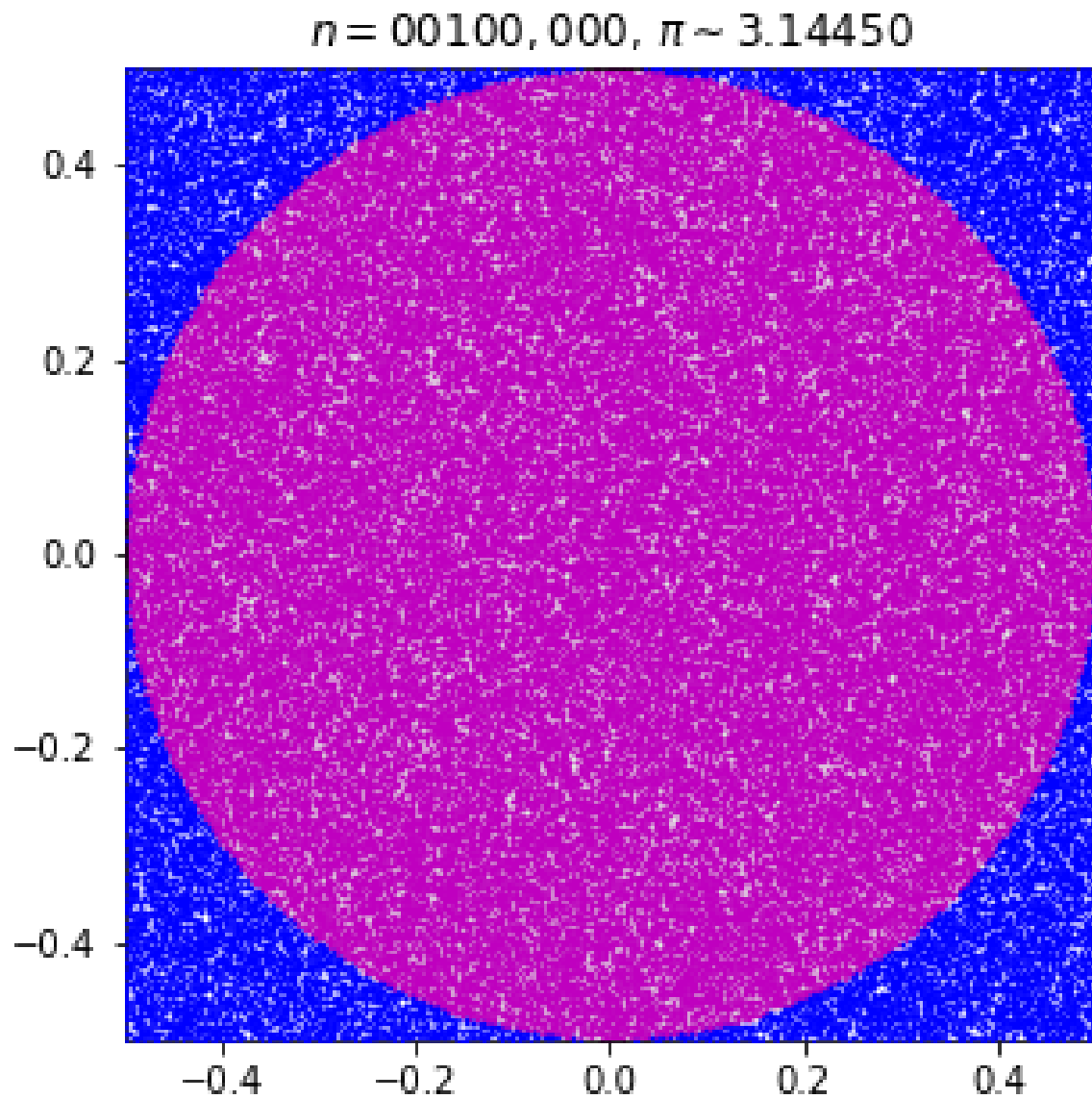


Figura 1.16: Monte Carlo estimation of π using random sampling within a square containing an inscribed circle.